Tesco Price Tracker

A comprehensive tool to track and visualize historical prices of products on the Tesco Hungary (HU) website. This project consists of a Python-based backend that scrapes and stores price history, and a seamless browser extension that displays immersive price charts right on the Tesco product pages.

Website / Original Repository: https://github.com/Smart123s/TescoPriceTracker

Authors: Smart123s and Bogat25

🚀 Key Features

- **Historical Price Tracking:** Continuously monitors product prices and generates historical data via a robust Python backend.
- **On-page Visualization:** Injects beautiful and responsive `Chart.js` graphs directly into Tesco's product pages through a browser extension.
- **Price Insights:** Calculates and displays useful metrics like Highest Price, Lowest Price, and Average Price.
- **Multi-language Support:** The extension automatically detects Tesco's page language (`en-HU` or `hu-HU`) to localize chart labels.
- **Automated & Scheduled:** Built-in Python scheduler to regularly scrape the Tesco GraphQL API and update local JSON records.4

🧩 Components

This project is split into two main operational parts:

1. Python Backend & Scraper (Data Collection)

The backend does the heavy lifting of gathering product data from the Tesco API, saving price histories to local JSON files (`data/` folder), and managing schedules.

- **`scraper.py`**: The core data gathering script. It iterates over the Tesco sitemap to find product URLs (`tpnc` codes) and queries the Tesco API for current prices and clubcard promotions.
- **`database_manager.py`**: Manages persistent storage. It reads and writes historical data to individual JSON files, ensuring only new or changed prices are appended to track trends.
- **`scheduler.py`**: A cron-style runner that triggers the scraper automatically matching a defined schedule.
- **`app.py`**: A Flask web application that serves as a frontend to search and view the tracked data locally.
- **`export.ps1`**: A PowerShell script automating the packaging of the browser extension into an installable `.zip` file inside the `versions/` folder.

2. Browser Extension (Frontend Display)

A Manifest V3 browser extension (compatible with Chrome, Edge, and Firefox) that superimposes the collected data as you shop.

- **Background Scripts**: Handles background operations and fetches the historical price JSONs for the current product.
- **Content Scripts**: Extracts the current product ID (TPNC) directly from the URL or DOM, calculates price stats, finds a safe insertion spot above the product description, and renders the `Chart.js` price history graph.
- **Popup UI**: A simple toggle interface to enable or disable the chart injection globally while remembering the user's preference.

🛠️ Installation & Usage

Setting up the Python Backend

1. Clone the repository to your local machine.
2. Ensure you have Python installed.
3. Install the required dependencies:

```
pip install -r requirements.txt
```

4. Set up your `.env` configuration (if applicable) for the API or scraper limits.
5. Run the web server or the scheduler:

```
python app.py
# OR for scraping
python scheduler.py
```

Installing the Browser Extension

To load the extension into your browser:

For Chrome / Edge:

1. Open your browser and navigate to `chrome://extensions/` (or `edge://extensions/`).
2. Enable **Developer mode** (usually a toggle in the top right corner).
3. Click **Load unpacked**.
4. Select the `extension/` folder from this repository.
5. Alternatively, run `.\export.ps1` in a PowerShell terminal, and extract or load the resulting `.zip` from the `versions/` folder.

For Firefox:

1. Navigate to `about:debugging#/runtime/this-firefox`.
2. Click **Load Temporary Add-on...**.
3. Select the `manifest.json` file inside the `extension/` folder.

🤝 Credits

- **Smart123s**: Co-Author
- **Bogat25**: Co-Author

For any issues, feature requests, or contributions, please visit the GitHub Repository.